创建型模式

简单工厂模式（非23种设计模式之一）

定义：简单工厂模式又称为静态工厂方法模式，它属于类创建型模式。专门定义一个类来负责创建其他类的实例，可以根据参数的不同返回不同类的实例，被创建的实例通常都具有共同的父类。

模式结构：Factory：工厂角色 Product：抽象产品角色 ConcreteProduct：具体的产品角色

模式动机：

> 1. 只需要知道参数的名字则可以得到相应的对象
> 2. 需要创建一些来自相同父类的类的实例
> 3. 当你需要什么，只需要传入一个正确的参数，就可以获取你所需要的对象，而无需知道其创建细节。

代码：

```
// 抽象产品类
public abstract class AbstractPay
{
  public abstract void pay();
}
// 具体产品类
public class CashPay extends AbstractPay{
  public void pay(){
    //现金支付方式处理代码  }
}
public class CreditcardPay extends AbstractPay{
  public void pay(){
    //信用卡支付方式处理代码 }
```

```
}
// 静态工厂类
public class PayMethodFactory
{
  public static AbstractPay getPayMethod(String type)•{
    if(type.equalsIgnoreCase(“cash”)){
            return new CashPay();//根据参数创建具体产品
     }
      else if(type.equalsIgnoreCase(“creditcard”)) {
          return new CreditcardPay();//根据参数创建具体产品

     }
    …
}
```

优点：

1. 实现了对象创建和使用的分离
2. 客户端无须知道所创建的具体产品类的类名，只需要知道具体产品类所对应的参数即可
3. 通过引入配置文件，可以在不修改任何客户端代码的情况下更换和增加新的具体产品类，在一定程度上提高了系统的灵活性

缺点：

1. 工厂类集中了所有产品的创建逻辑，职责过重，一旦不能正常工作，整个系统都要受到影响
2. 增加系统中类的个数，增加了系统的复杂度和理解难度
3. 系统扩展困难，一旦添加新产品不得不修改工厂逻辑
4. 由于使用了静态工厂方法，造成工厂角色无法形成基于继承的等级结构，工厂类不能得到很好地扩展

适用环境：

> 1. 工厂类负责创建的对象比较少，由于创建对象比较少，不会造成工厂方法中的逻辑太过复杂。
> 2. 客户端只知道传入工厂类的参数，对于如何创建对象不关心：客户端既不需要关心创建细节，甚至于连类名都不需要记住，只需要知道类型所对应的参数即可。

工厂方法模式

回顾：简单工厂模式的不足（加入新产品时，必须修改工厂）

定义：简称工厂模式，也叫虚拟构造器(Virtual Constructor)模式或者多态工厂(Polymorphic Factory)模式，它属于类创建型模式。在工厂方法模式中，工厂父类负责定义创建产品对象的公共接口， 而工厂子类则负责生成具体的产品对象，这样做的目的是将产品类的实例化操作延迟到工厂子类中完成，即通过工厂子类来确定 究竟应该实例化哪一个具体产品类。

模式结构： 抽象产品类 具体产品类 抽象工厂类 具体工厂类

模式动机：简单工厂模式的不足（加入新产品时，必须修改工厂）不满足开闭原则

代码：

```
// 抽象产品类
public abstract class AbstractPay
{
  public abstract void pay();
}
// 具体产品类
public class CashPay extends AbstractPay{
  public void pay(){
    //现金支付方式处理代码  }
}
public class CreditcardPay extends AbstractPay{
  public void pay(){
    //信用卡支付方式处理代码 }
}
// 抽象工厂类
```

```
public abstract class PayMethodFactory{
  public abstract AbstractPay getPayMethod( );
}
// 具体工厂类
public class CashPayFactory extends PayMethodFactory{
  public AbstractPay getPayMethod(){
    return new CashPay();
    }
}
```

优点：

1. 工厂方法用来创建客户所需要的产品，同时还向客户隐藏了哪种具体产品类将被实例化这一细节
2. 能够让工厂自主确定创建何种产品对象，而如何创建这个对象的细节则完全封装在具体工厂内部
3. 在系统中加入新产品时，完全符合开闭原则

缺点：

1. 系统中类的个数将成对增加，在一定程度上增加了系统的复杂度，会给系统带来一些额外的开销
2. 增加了系统的抽象性和理解难度

适用环境：

1. 客户端不知道它所需要的对象的类（客户端不需要知道具体 产品类的类名，只需要知道所对应的工厂即可，具体产品对 象由具体工厂类创建）
2. 抽象工厂类通过其子类来指定创建哪个对象

抽象工厂模式

定义：提供一个创建一系列相关或相互依赖对象的接口，而无须指定它们具体的类。抽象工厂模式又称为Kit模式，属于对象创建型模式。

模式动机：回顾工厂方法模式，当具体产品增加时，系统中类的个数将成对增加，在一定程度上增加了系统的复杂性。因此我们需要设计出一个工厂可以生产多个产品对象。

模式结构：AbstractFactory：抽象工厂类 ConcreteFactory：具体工厂 AbstractProduct：抽象产品 ConcreteProduct：具体产品

代码：

```
//抽象工厂类
public abstract class AbstractFactory
{
    public abstract AbstractProductA createProductA( );
    public abstract AbstractProductB createProductB( );
}
// 具体工厂之一
public class ConcreteFactory1 extends  AbstractFactory
{
   public AbstractProductA createProductA( ) {
    return new ConcreteProductA1();
     }
     public AbstractProductB createProductB( ) {
    return new ConcreteProductB1();
     }
}
```

优点：

1. 隔离了具体类的生成，使得客户端并不需要知道什么被创建
2. 当一个产品族中的多个对象被设计成一起工作时，它能够保证客户端始终只使用同一个产品族中的对象

> 3. 增加新的产品族很方便，无须修改已有系统，符合开闭原则

缺点：

> 1. 增加新的产品等级结构麻烦，需要对原有系统进行较大的修改，甚至需要修改抽象层代码，这显然会带来较大的不便，违背了开闭原则

适用环境：

> 1. 一个系统不应当依赖于产品类实例如何被创建、组合和表达的细节
> 2. 系统中有多于一个的产品族，但每次只使用其中某一产品族
> 3. 属于同一个产品族的产品将在一起使用，这一约束必须在系统的设计中体现出来
> 4. 产品等级结构稳定，在设计完成之后不会向系统中增加新的产品等级结构或者删除已有的产品等级结构

三种工厂模式对比

简单工厂模式

优点：很明显，简单工厂的特点就是“简单粗暴”，通过一个含参的工厂方法，我们可以实例化任何产品类，上至飞机火箭，下至土豆面条，无所不能。所以简单工厂有一个别名：上帝类。

缺点：任何“东西”的子类都可以被生产，负担太重。当所要生产产品种类非常多时，工厂方法的代码量可能会很庞大。在遵循开闭原则（对拓展开放，对修改关闭）的条件下，简单工厂对于增加新的产品，无能为力。因为增加新产品只能通过修改工厂方法来实现。

工厂方法模式

优点：工厂方法模式就很好的减轻了工厂类的负担，把某一类/某一种东西交由一个工厂生产；（对应简单工厂的缺点1）同时增加某一类”东西“并不需要修改工厂类，只需要添加生产这类”东西“的工厂即可，使得工厂类符合开放-封闭原则。

缺点：相比简单工厂，实现略复杂。对于某些可以形成产品族的情况处理比较复杂。

抽象工厂模式

优点：针对产品族场景，实现比较工整；

缺点：比较复杂，只适用于产品族场景

建造者模式

定义：是最复杂的创建型模式，它用于创建一个包含多个组成部分的复杂对象，可以返回一个完整的产品对象给用户。将一个复杂对象的构建与它的表示分离，使得同样的构建过程可以创建不同的表示。

动机：创建复杂对象

模式结构：Product：产品类 Director：指导者类 Builder：抽象建造者类 ConcreteBuild：具体建造者

代码：

```
// Director：指导者类
public class Director {
    private Builder builder;

    public Director(Builder builder) {
        this.builder=builder;
    }

    public void setBuilder(Builder builder) {
        this.builder=builer;
    }
  //产品构建与组装方法
    public Product construct() {
        builder.buildPartA();
```

```
        builder.buildPartB();
        builder.buildPartC();
        return builder.getResult();
    }
}
// Builder：抽象建造者类
public abstract class Builder {
    //创建产品对象
    protected Product product=new Product();
    public abstract void buildPartA();
    public abstract void buildPartB();
    public abstract void buildPartC();

    //返回产品对象
    public Product getResult() {
        return product;
    }
}
// ConcreteBuild：具体建造者
public class ConcreteBuilder1 extends Builder{
    public void buildPartA() {
        product.setPartA("A1");
    }
    public void buildPartB() {
        product.setPartB("B1");
    }
    public void buildPartC() {
        product.setPartC("C1");
    }
}
// Product产品类
public class Product {
  private String partA; //定义部件，部件可以是任意类型，包括值类型和引用类型
  private String partB;
  private String partC;

```

```
    //partA的Getter方法和Setter方法省略
    //partB的Getter方法和Setter方法省略
    //partC的Getter方法和Setter方法省略
}
```

优点：

1. 客户端不必知道产品内部组成的细节，将产品本身与产品的创建过程解耦，使得相同的创建过程可以创建不同的产品对象
2. 每一个具体建造者都相对独立，与其他的具体建造者无关，因此可以很方便地替换具体建造者或增加新的具体建造者，扩展方便，符合开闭原则
3. 可以更加精细地控制产品的创建过程

缺点：

1. 建造者模式所创建的产品一般具有较多的共同点，其组成部分相似，如果产品之间的差异性很大，不适合使用建造者模式，因此其使用范围受到一定的限制
2. 如果产品的内部变化复杂，可能会需要定义很多具体建造者类来实现这种变化，导致系统变得很庞大，增加了系统的理解难度和运行成本

适用环境：

1. 需要生成的产品对象有复杂的内部结构，这些产品对象通常包含多个成员变量
2. 需要生成的产品对象的属性相互依赖，需要指定其生成顺序
3. 对象的创建过程独立于创建该对象的类。在建造者模式中通过引入了指挥者类，将创建过程封装在指挥者类中，而不在建造者类和客户类中
4. 隔离复杂对象的创建和使用，并使得相同的创建过程可以创建不同的产品

原型模式

定义：原型模式是一种对象创建型模式，用原型实例指定创建对象的种类，并且通过复制这些原型创建新的对象。原型模式允许通过一个原型对象创建一个或多个同类型的其他对象，而无须知道任何创建的细节

动机：有些对象的创建过程较为复杂，而且需要频繁创建，通过给出一个原型对象来指明所要创建的对象的类型，然后用复制这个原型对象的办法创建出更多同类型的对象。

浅克隆：当原型对象被复制时，只复制它本身和其中包含的值类型的成员变量，而引用类型的成员变量并没有复制

深克隆：除了对象本身被复制外，对象所包含的所有成员变量也将被复制

模式结构：Client：客户类 Prototype：抽象原型类

代码：

```
// 能够实现克隆的Java类必须实现一个标识接口Cloneable，表示这个Java类支持复制
// Cloneable接口和Serializable接口其代码都非常简单，它们是空接口。这种空接口也称为标识接口
public class PrototypeDemo implements Cloneable
{
    ……
    public Object clone()
    {
        Object object = null;
        try {
            object = super.clone( );
        } catch (CloneNotSupportedException exception) {
            System.err.println("Not support cloneable");
        }
        return object;
    }
    ……
```

```
17    }
```

优点：

1. 简化对象的创建过程，通过复制一个已有实例可以提高新实例的创建效率
2. 扩展性较好
3. 简化创建结构，原型模式中产品的复制是通过封装在原型类中的克隆方法实现的，无须专门的工厂类来创建产品
4. 可以使用深克隆的方式保存对象的状态，以便在需要的时候使用，可辅助实现撤销操作

缺点：

1. 需要为每一个类配备一个克隆方法，而且该克隆方法位于一个类的内部，当对已有的类进行改造时，需要修改源代码，违背了开闭原则。
2. 在实现深克隆时需要编写较为复杂的代码，而且当对象之间存在多重的嵌套引用时，为了实现深克隆，每一层对象对应的类都必须支持深克隆，实现起来可能会比较麻烦。

适用环境：

1. 创建新对象成本较大，新对象可以通过复制已有对象来获得，如果是相似对象，则可以对其成员变量稍作修改
2. 系统要保存对象的状态，而对象的状态变化很小
3. 需要避免使用分层次的工厂类来创建分层次的对象

单例模式

定义： 确保某一个类只有一个实例，而且自行实例化并向整个系统提供这个实例，这个类称为单例类，它提供全局访问的方法。

动机：为了确保一个类只有一个实例并且这个实例易于被访问，确保一个类只有一个实例并且这个实例易于被访问，更好的办法是让类自身负责创建和保存它的唯一实例，并保证不能创建其他实例，并且提供一个访问该实例的方法。（机制实现）

模式结构：Singleton：单例类

代码：

```
public class Singleton {
      //静态私有成员变量
    private static Singleton instance=null;
        //私有构造函数
  private Singleton() {
  }
       //静态公有工厂方法，返回唯一实例
  public static Singleton getInstance() {
    if(instance==null)
        instance=new Singleton();
    return instance;
  }
}

// 饿汉式
public class EagerSingleton {
    private static final EagerSingleton instance = new
EagerSingleton();
    private EagerSingleton() { }
    public static EagerSingleton getInstance() {
        return instance;
    }
}

//懒汉式
public class LazySingleton {
  private static LazySingleton instance = null;
  private LazySingleton() { }

```

```
  public static LazySingleton getInstance() {
    if (instance == null) {
      instance = new LazySingleton();
     }
    return instance;

  }
}

// 懒汉式有线程不安全

// 解决方法：锁方法
public class LazySingleton {
  private static LazySingleton instance = null;
  private LazySingleton() { }
  synchronized public static LazySingleton getInstance() {
      if (instance == null) {
        instance = new LazySingleton();
      }
      return instance;
  }
}

// 锁代码段
public static LazySingleton getInstance() {
    if (instance == null) {
        synchronized (LazySingleton.class) {
            instance = new LazySingleton();
        }
    }
    return instance;
}

// 双重判断检验
public class LazySingleton {
    private volatile static LazySingleton instance = null;
    private LazySingleton() { }
```

```
65      public static LazySingleton getInstance() {
66          //第一重判断
67          if (instance == null) {
68              //锁定代码块
69              synchronized (LazySingleton.class) {
70                  //第二重判断
71                  if (instance == null) {
72                      instance = new LazySingleton(); //创建单例实例
73                  }             }         }
74      return instance;
75      }
76  }
```

优点：

1. 提供了对唯一实例的受控访问
2. 可以节约系统资源，提高系统的性能
3. 允许可变数目的实例（多例类）

缺点：

1. 扩展困难（缺少抽象层）
2. 单例类的职责过重。单例模式与单一职责原则有冲突。一个类应该只实现一个逻辑，而不关心它是否是单例的，是不是要单例取决于环境，单例模式把“要单例”和业务逻辑融合在一个类中
3. 由于自动垃圾回收机制，可能会导致共享的单例对象的状态丢失

适用环境：

1. 系统只需要一个实例对象，或者因为资源消耗太大而只允许创建一个对象
2. 客户调用类的单个实例只允许使用一个公共访问点，除了该公共访问点，不能通过其他途径访问该实例